Data Race

- Distinct threads access a memory location
- At least one of the accesses modifies the location
- At least one of the accesses is non-atomic
- The accesses are not ordered by synchronization

Fairness Pros:

- Avoid starvation
- Avoid deadlock

Cons:

- Cache waste: Suppose lock grants access to shared resource. Threads getting access again means resources are still in cache. Fair locks cause each thread to update cache.
- If thread count > processor count, to ensure fairness, there can be huge context switch overhead.

Spinlocks

- Consumes CPU
- No system calls
- - Good if low contention or short critical sections

Atomic CAS

- compare_exchange_strong returns false only if we lose a race against another thread – another thread must have changed the value.
- compare_exchange_weak can fail spuriously – it may return false even if the field value matches the expected value, and no other thread is modifiying (fail even if no race).
- fetch_add: return old value, then add

Performance Problem of exchange

When thread A is holding the lock, thread B exchange(true) always returns true, invalidating all other threads' caches, hence wastes memory bandwidth.

Relaxed Consistency

SC only on same location.

- Relaxed load: you may not see other threads' updates.
- Relaxed store: your updates may not be updated to other threads.

Spinning using relaxed is wrong.

Release-acquire Consistency

- Release: When a thread releases a lock, it effectively announces that the data modified by that thread up to that point is complete and consistent. This release operation ensures that all prior writes to shared data are visible to other threads.
- Acquire: Conversely, when a thread acquires a lock, it is made aware of all the data modified and released by other threads. The acquire operation ensures that the thread sees a consistent view of the memory.
- Release-acquire: all previous writes are visible to other threads, and all previous writes are visible to this thread.

Spinlock: Active Backoff

```
while (lock_bit_.exchange(true)) {
  do {
    // Solution 1: volatile
    for (volatile size_t i = 0; i < 100; i++) {
    }

    // Solution 2: Intel
    for (size_t i = 0; i < 100; i++) {
        __mm_pause();
    }
  } while (lock_bit_.load());
}
```

Spinlock: Exponential Backoff

Backoff can underutilise critical section when lock is contended

Ticket Lock

- The lock stores next_ticket and now_serving
- Thread gets the lock only if no_serving is the ticket it is holding. Then the thread releases a lock, now_serving++.
- If currently unlocked, next_ticket = now_serving
- If locked, new threads get next_ticket++

Problems:

- When all processes have the same access to memory, it is fair. But on a non-uniform memory architecture (NUMA) system, not all processes have equal access to locks. Processes on the same NUMA node as the lock have an unfair advantage in obtaining the lock.

Cons:

- If thread count > processor count, then threads are to be scheduled, then this simple ticket lock will result in huge context switch overhead and perform extremely badly.

Futex

- Futex can be used to sync threads in multiple processes.

FUTEX_WATE(int* p, int v):

- Returns immediately if *p != v
- Otherwise, add thread to wait queue associated with p (does not access the address, just use the address)

FUTEX_WAKE(int* p, int wake_count):

- Wake up wake_count threads that are on the wait queue for p
- 1 and INT_MAX: the only sensible values for wake_count

Use a atomic<int> state_ for the p. State_ is 0: lock is free; 1: lock is held.

Locking: Atomically exchange state_ with 1.

- If result is 0, we got the mutex, return.
- Otherwise loop sleep via FUTEX_WAIT until get 1.

Unlocking:

- Store 0 to state_.
- Wake one thread using FUTEX_WAKE.

Haskell

- newMVar :: a -> IO (MVar a)
- newEmptyMVar :: IO (MVar a)
- readMVar :: MVar a -> IO a: Block until the MVar is full, retrieve it, and leave it full.
- takeMVar :: MVar a -> IO a: Block until the MVar is full, retrieve it, and make it empty.
- putMVar :: MVar a -> a -> IO (): Block until MVar is empty, and write value into MVar.

Rust

```
let ar: Arc<AtomicU32> = Arc::new(AtomicU32::new(
ar_t1.fetch_add(temp, Ordering::Relaxed);

let r1: Mutex<u32> = Mutex::new(0);
let r1_arc: Arc<Mutex<u32>> = Arc::new(result1);
let r1_arc_for_t = result1_arc.clone();
*r1_arc_for_t.lock().unwrap() = my_result;
```

Vector Clock

- Initial: C: each thread see itself is 1, others are 0. Table header row: C_0, L_m, R_x, W_x, etc.
- Acquire: Update thread's vector clock to be: column-wise-max(itself, lock's state).
- Read: If $\boldsymbol{W_x} \sqsubseteq \boldsymbol{C_t}$ then update $\boldsymbol{R_x}(t)$ to current thread's **logical clock** (not the whole vector clock), otherwise race.
- Write: If $\boldsymbol{W_x} \sqsubseteq \boldsymbol{C_t}$ and $\boldsymbol{R_x} \sqsubseteq \boldsymbol{C_t}$ then update $\boldsymbol{W_x}(t)$ to current thread's **logical clock** (not the whole vector clock), otherwise race.
- Release lock M: $\boldsymbol{L_m} \leftarrow \boldsymbol{C_t}$ and increment $\boldsymbol{C_t}(t)$.

Safety: Something bad will never happen. If it is violated, it is done by a finite computation.

- Both threads never get the lock simultaneously.

Liveness: Something good will eventually happen. Cannot be violated by a finite computation (intuition: we can always run longer and see what happens)

- Anything can go wrong will go wrong.
- No deadlock.

Amdahl's Law: Speedup $= \frac{\text{1 thread exec time}}{\text{n thread exec time}} = \frac{1}{1-p+\frac{p}{n}}$

TSO: allows the following for adjacent instructions (in program order)

- write-read reordering on different locations (SB)
- write-to-read forwarding on the same location

- write-assignment reordering
- **NO** write-write reordering

PSO: Allows write-write reordering on different locs.
sfence: write-write reordering 不能越过 sfence. 在 sfence 之前的 write, 一定会在 sfence 后面的 write 之前 commit.
x, y, z are memory locations; a, b, c are registers.
TSO-trace: $P_{sb}, S_0, M_0, B_0 \to P_1, S_0, M_0, B_1$ with $P_1 = \ldots$
$x := 1 \prec a := x$: …before …
Write SB: $P_{sb}, S_0, M_0, B_0 \to P_1, S_0, M_0, B_1$ with $P_1 = P[\tau_1 \mapsto \text{skip}; a := y], B_1 = B_0[\tau_1 \mapsto (W, x, 1)]$
Write Reg: $\to P_2, S', M_0, B_1$ with $P_2 = P_1[\tau_1 \mapsto \text{skip}], S' = S_0[\tau_1 \mapsto s1], s_1 = s_0[a \mapsto 0]$
Commit SB: $\to P_{\text{skip}}, S, M_1, B_3$ with $M_1 = M_0[x \mapsto 1], B_3 = B_2[\tau_1 \mapsto \emptyset]$
P skip: $\to P_{\text{skip}}, S, M_0, B_2$ with …
SC-Outcome (M, S): $M = M_0[x \mapsto 1, y \mapsto 1]$, $S_1 = S_0[\tau_2 \mapsto s_1]$ with $s_1 = s_0[a \mapsto 0, b \mapsto 0]$

SC/TSO Sequential Transitions

$$\frac{s' = s[a \mapsto v]}{a := x, s \xrightarrow{(\mathtt{R},x,v)}_{\mathsf{c}} \mathtt{skip}, s'}$$ **(a)** Seq-Mem-Read

$$\frac{s(a) = v}{x := a, s \xrightarrow{(\mathtt{W},x,v)}_{\mathsf{c}} \mathtt{skip}, s}$$ **(b)** Seq-Mem-Write

$$\frac{\mathsf{eval}(s, E) = v \qquad v_n = v_o + v}{\mathtt{FAA}(x, E), s \xrightarrow{(\mathtt{U},x,v_o,v_n)}_{\mathsf{c}} \mathtt{skip}, s}$$ **(c)** Seq-Fetch-Add-Assign

$$\frac{\mathsf{eval}(s, E_o) = v_o \qquad \mathsf{eval}(s, E_n) = v_n \qquad s' = s[a \mapsto 1]}{a := \mathtt{CAS}(x, E_o, E_n), s \xrightarrow{(\mathtt{U},x,v_o,v_n)}_{\mathsf{c}} \mathtt{skip}, s'}$$ **(a)** Seq-CAS-Success

$$\frac{\mathsf{eval}(s, E_o) = v_o \qquad \mathsf{eval}(s, E_n) = v_n \qquad s' = s[a \mapsto 1]}{a := \mathtt{CAS}(x, E_o, E_n), s \xrightarrow{(\mathtt{U},x,v_o,v_n)}_{\mathsf{c}} \mathtt{skip}, s'}$$ **(b)** Seq-CAS-Fail

SC Storage Transitions

$$\frac{M(x) = v}{M \xrightarrow{\tau:(\mathtt{R},x,v)}_{\mathsf{m}} M}$$ **(a)** Mem-Read

$$\frac{M(x) = v_o \qquad M' = M[x \mapsto v_n]}{M \xrightarrow{\tau:(\mathtt{U},x,v_o,v_n)}_{\mathsf{m}} M'}$$ **(b)** CAS-Success

$$\frac{M(x) = v}{M \xrightarrow{\tau:(\mathtt{U},x,v,\perp)}_{\mathsf{m}} M}$$ **(c)** CAS-Fail

TSO sequential transition

$$\frac{B(\tau) = b \qquad \mathsf{get}(M, b, x) = v}{M, B \xrightarrow{\tau:(\mathtt{R},x,v)}_{\mathsf{m}} M, B} \qquad \mathsf{get}(M, b, x) \triangleq \begin{cases} v & \text{if } \exists b_1, b_2.\ b = b_1.(\mathtt{W}, x, v).b_2 \\ & \wedge\ \neg\exists v'.\ (\mathtt{W}, x, v') \in b_2 \\ M(x) & \text{otherwise} \end{cases}$$

(a) TSO-Read

$$\frac{B(\tau) = b \qquad b' = b.(\mathtt{W}, x, v) \qquad B' = B[\tau \mapsto b']}{M, B \xrightarrow{\tau:(\mathtt{W},x,v)}_{\mathsf{m}} M, B'}$$ **(b)** TSO-Write

$$\frac{B(\tau) = \emptyset \qquad M(x) = v_o \qquad M' = M[x \mapsto v_n]}{M, B \xrightarrow{\tau:(\mathtt{U},x,v_o,v_n)}_{\mathsf{m}} M', B}$$ **(a)** TSO-RMW-Success

$$\frac{B(\tau) = \emptyset \qquad M(x) = v_o \qquad M' = M[x \mapsto v_n]}{M, B \xrightarrow{\tau:(\mathtt{U},x,v_o,v_n)}_{\mathsf{m}} M', B}$$ **(b)** TSO-RMW-Fail

$$\frac{B(\tau) = \emptyset}{M, B \xrightarrow{\tau:\mathtt{MF}}_{\mathsf{c}} M, B}$$ **(a)** MFence

$$\frac{B(\tau) = (\mathtt{W}, x, v).b \qquad M' = M[x \mapsto v] \qquad B' = B[\tau \mapsto b]}{M, B \xrightarrow{\tau:\epsilon}_{\mathsf{m}} M', B'}$$ **(b)** TSO-Propagation of Delayed Writes

Sequential Specification: Sequential objects are if-then-else, i.e. precondition-postcondition(result)-postcondition(state).
Sequential vs Concurrent: In Seq each method call is atomic, methods described in isolation.
Pending: no matching response.
Well-Formed Histories: Per-thread invocations are sequential.
Equivalent Histories: Per-thread invocations (H|A) are the same.
Legal Sequential History: A sequential (multi-object) history H is legal if: for every object x, H|x is in the sequential specification for x.
Precedence: $m_0 \to_H m_1$: A method call precedes another if response event precedes invocation event.
Correct Execution: Execution is correct if this sequential behaviour is correct.
Linearizable:
History H is linearizable if it can be extended to G by

- appending zero or more responses to pending invocations those take effect,
- discarding other pending invocations.

So that G is equivalent to **legal sequential history** S, where $\to_G \subseteq \to_S$.
Note:

- Linearizability is a note of **correctness**.

Linearizable Object: An object is linearizable if all of its possible executions are linearizable.
Sequential Consistency: History H is sequentially consistent if same as linearizable except for $\to_G \subseteq \to_S$ does not need to hold (program order is not preserved). That is, **no need to preserve real-time order**:

- Cannot re-order operations done by the same thread
- **Can re-order non-overlapping operations done by different threads**

Shortcuts

- Linearizability is strictly stronger than sequential consistency.
- If H is not sequentially consistent, it is also not linearizable.
- If H is not linearizable, it may be sequentially consistent.

Composability History H is linearizable if and only if: for every object x, H|x is linearizable.
Progress Conditions

- Deadlock-free/Starvation-free: some/every thread trying to acquire the lock eventually succeeds.
- Lock-free/Wait-free: some/every thread calling a method eventually returns.

Proving Non-Linearize

1. Assume H is Linearizable, then there exists a legal sequential history S, such that
2. G is equivalent to S, and that $\to_G \subseteq \to_S$
3. Prove S is not legal.

Proving Linearizable

1. Name threads …
2. We can represent history H formally as.
3. Given the composability of linearisability, it suffices to show that H|x is linearisable and H|y is linearisable: (define H|x and H|y)
4. Define a sequential history S_x (imagination)
5. First show H|x is linearizable.
6. (4) Obtain G_x by discarding the pending invocation $x.read(3)$ (define G_x)
7. We then have: 'Gx|A = Sx|A = A r.read(), A r:1', …, and thus (1) G_x and S_x are equivalent.
8. Moreover, (2) S_x is a legal, sequential history of memory location objects.
9. (3) Show $\to_H \subseteq \to_S$ (precedence):
$\to_H = \{r.read(1) \to r.read(2), r.write(1) \to r.read(2), r.write($
$r.read(2)\}$
10. From (1) (2) (3), H is linearizable.

Proving Sequential Consistency

1. We can represent history H formally as.
2. Imagine a history S that is a legal sequential history.
3. State H|x = S|A = A r.read(), A r:1, …
4. and thus H and S are equivalent.
5. Since H is a complete history, from 1 and 2 we know H is sequentially consistent.

Proving Non-Sequential Consistency
Let us proceed by contradiction and assume H is sequentially consistent. As H is already complete, this means there exists a sequential history S such that (6) S is a legal history and (7) S is equivalent to H.
For S to be a legal history as per (6) we must have:

- $x.push(b) \to_S x.push(a) \to_S x.pop(a)$(8)
- $y.push(b) \to_S y.push(a) \to_S y.pop(a)$(9)

On the other hand, from (7) and the definition of H we have:

- $y.pop(a) \to_S x.push(a)$(10)
- $x.pop(a) \to_S y.push(a)$(11)

Consequently, from (8), (9), (10) and (11) we have:

- $y.push(a) \to_S y.pop(a) \to_S x.push(a) \to_S x.pop(a) \to_S y.push(a)$(12)

That is, $y.push(a) \to_S y.push(a)$ and thus $\to_S$ contains a cycle. This, however, leads to a contradiction as S is a sequential history and thus $\to_S$ must be a strict (irreflexive) total order.